



Universidad Don Bosco

Guía de ejercicios #2

POO – G02T

- William Adonay Bonilla Pérez BP220765
- Ingeniero Juan Carlos Menjívar Ramírez
- Técnico en Ingeniería en Computación
- 30 de septiembre de 2025
- Enlace del repositorio:
<https://github.com/LiamXG04/Proyectos.git>

Introducción

El desarrollo de este proyecto tuvo como objetivo poner en práctica los principios fundamentales de la **programación orientada a objetos en Java (POO)**, a través de una serie de ejercicios progresivos que abarcan desde conceptos básicos como clases y objetos, hasta la implementación de pequeños sistemas integradores.

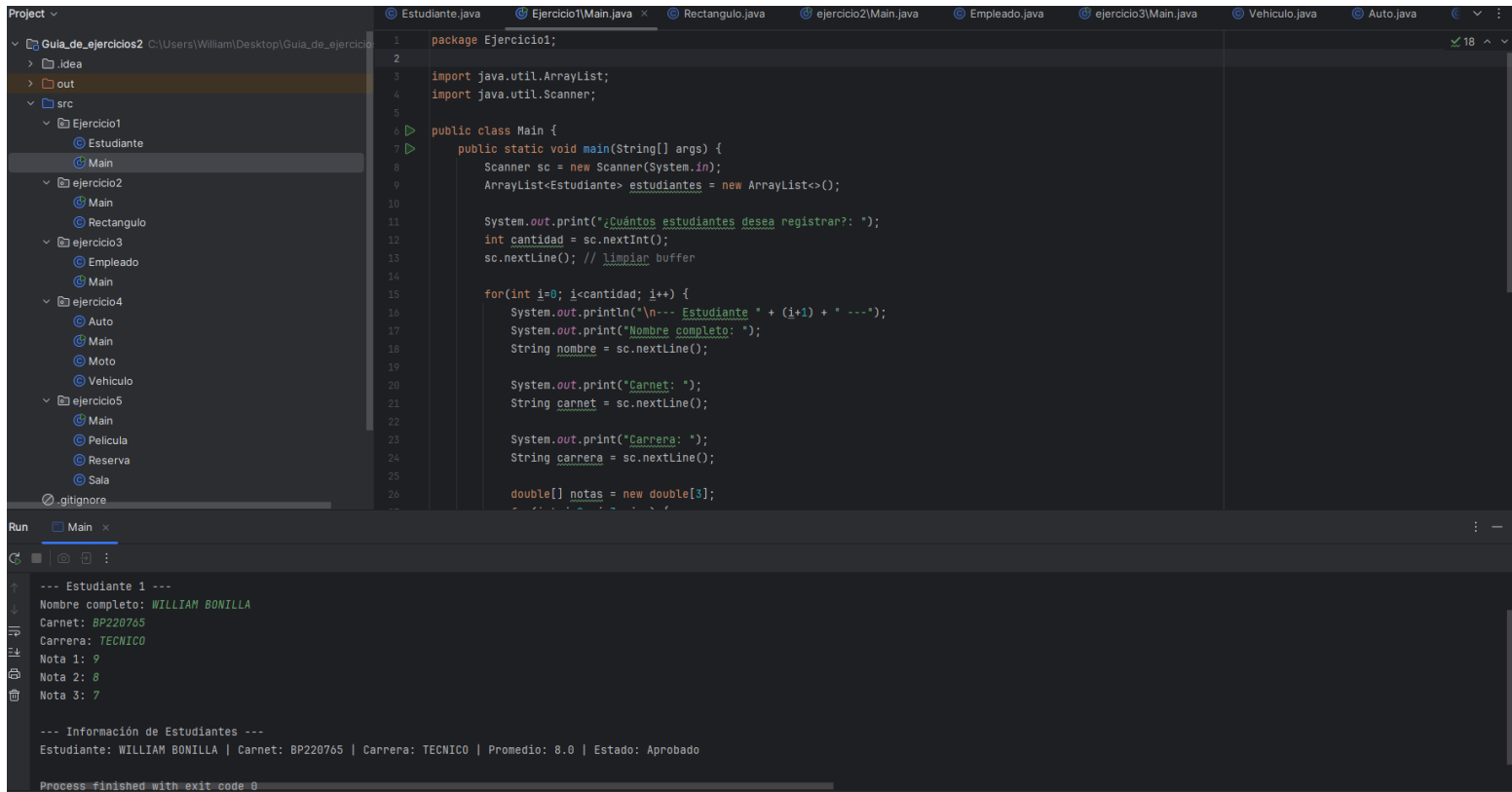
A lo largo de los ejercicios fuimos aplicando elementos esenciales como **encapsulamiento, constructores, sobrecarga de métodos, colecciones, herencia y polimorfismo**, todos pilares de la programación orientada a objetos. Por ejemplo, en el **Ejercicio 1** trabajamos con la clase **Estudiante**, aprendiendo a crear atributos privados, métodos **getters** y **setters**, además de cálculos automáticos como promedios y validación de aprobados o reprobados. Posteriormente, en el **Ejercicio 2**, reforzamos el uso de **constructores** y **sobrecarga**, creando figuras geométricas (**rectángulo** y **cuadrado**) para calcular su área y perímetro.

Más adelante, el **Ejercicio 3** nos llevó a la gestión de empleados con el uso de **ArrayList**, agregando operaciones más completas como agregar, listar, buscar y eliminar registros, lo cual se acerca mucho a escenarios de aplicaciones reales. En el **Ejercicio 4**, trabajamos con **herencia y modelado UML**, implementando la jerarquía de vehículos con autos y motos, lo cual permitió reforzar cómo funciona la reutilización de código y la especialización de clases. Finalmente, en el **Ejercicio 5** desarrollamos un mini sistema integrador de **reservas de cine**, que reunió todo lo aprendido: clases con **atributos** bien definidos, manejo de colecciones para almacenar datos, validaciones de capacidad y operaciones de agregar, cancelar y mostrar reservas.

En conjunto, estos ejercicios nos ayudan un poco a comprender los conceptos teóricos, sino también aplicarlos de manera práctica en la solución de problemas concretos. Cada programa refleja un paso en la construcción del pensamiento lógico, la organización del código y el diseño de soluciones basadas en objetos, tal como se espera en proyectos de mayor escala en el mundo real.

Capturas de pantalla y código

EJERCICIO 1



The screenshot shows an IDE with a project named 'Guia_de_ejercicios2'. The 'src' folder contains several subfolders, including 'Ejercicio1'. The 'Main' class in 'Ejercicio1' is selected, showing the following code:

```
1 package Ejercicio1;
2
3 import java.util.ArrayList;
4 import java.util.Scanner;
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         ArrayList<Estudiante> estudiantes = new ArrayList<>();
10
11         System.out.print("¿Cuántos estudiantes desea registrar?: ");
12         int cantidad = sc.nextInt();
13         sc.nextLine(); // limpiar buffer
14
15         for(int i=0; i<cantidad; i++) {
16             System.out.println("\n--- Estudiante " + (i+1) + " ---");
17             System.out.print("Nombre completo: ");
18             String nombre = sc.nextLine();
19
20             System.out.print("Carnet: ");
21             String carnet = sc.nextLine();
22
23             System.out.print("Carrera: ");
24             String carrera = sc.nextLine();
25
26             double[] notas = new double[3];
```

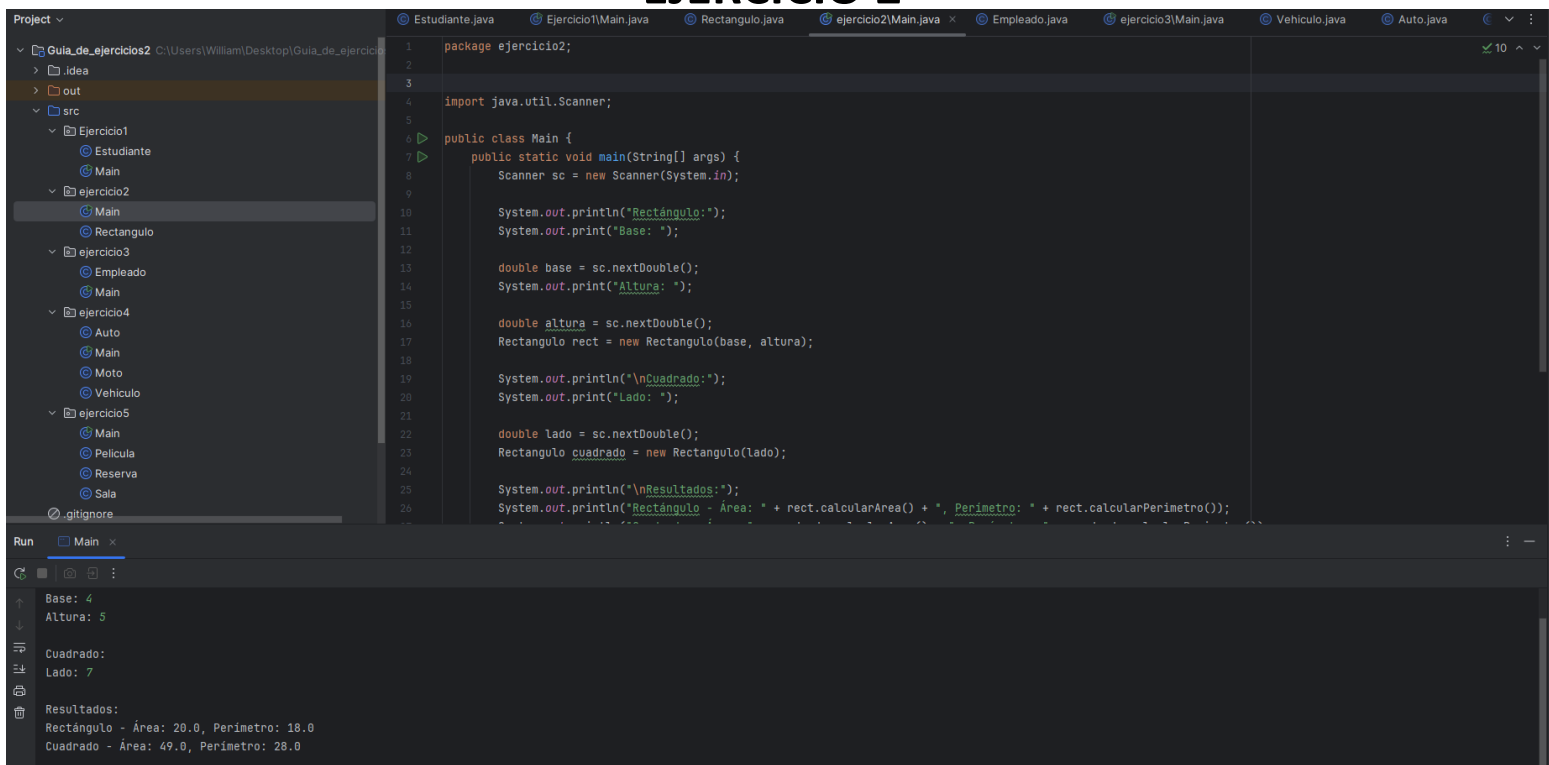
The 'Run' window shows the output of the program:

```
--- Estudiante 1 ---
Nombre completo: WILLIAM BONILLA
Carnet: BP220765
Carrera: TECNICO
Nota 1: 9
Nota 2: 8
Nota 3: 7

--- Información de Estudiantes ---
Estudiante: WILLIAM BONILLA | Carnet: BP220765 | Carrera: TECNICO | Promedio: 8.0 | Estado: Aprobado

Process finished with exit code 0
```

EJERCICIO 2



The screenshot shows an IDE with a project named 'Guia_de_ejercicios2'. The 'src' folder contains several subfolders, including 'Ejercicio2'. The 'Main' class in 'Ejercicio2' is selected, showing the following code:

```
1 package ejercicio2;
2
3
4 import java.util.Scanner;
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9
10        System.out.println("Rectángulo:");
11        System.out.print("Base: ");
12
13        double base = sc.nextDouble();
14        System.out.print("Altura: ");
15
16        double altura = sc.nextDouble();
17        Rectangulo rect = new Rectangulo(base, altura);
18
19        System.out.println("\nC cuadrado:");
20        System.out.print("Lado: ");
21
22        double lado = sc.nextDouble();
23        Rectangulo cuadrado = new Rectangulo(lado);
24
25        System.out.println("\nResultados:");
26        System.out.println("Rectángulo - Área: " + rect.calcularArea() + ", Perímetro: " + rect.calcularPerimetro());
```

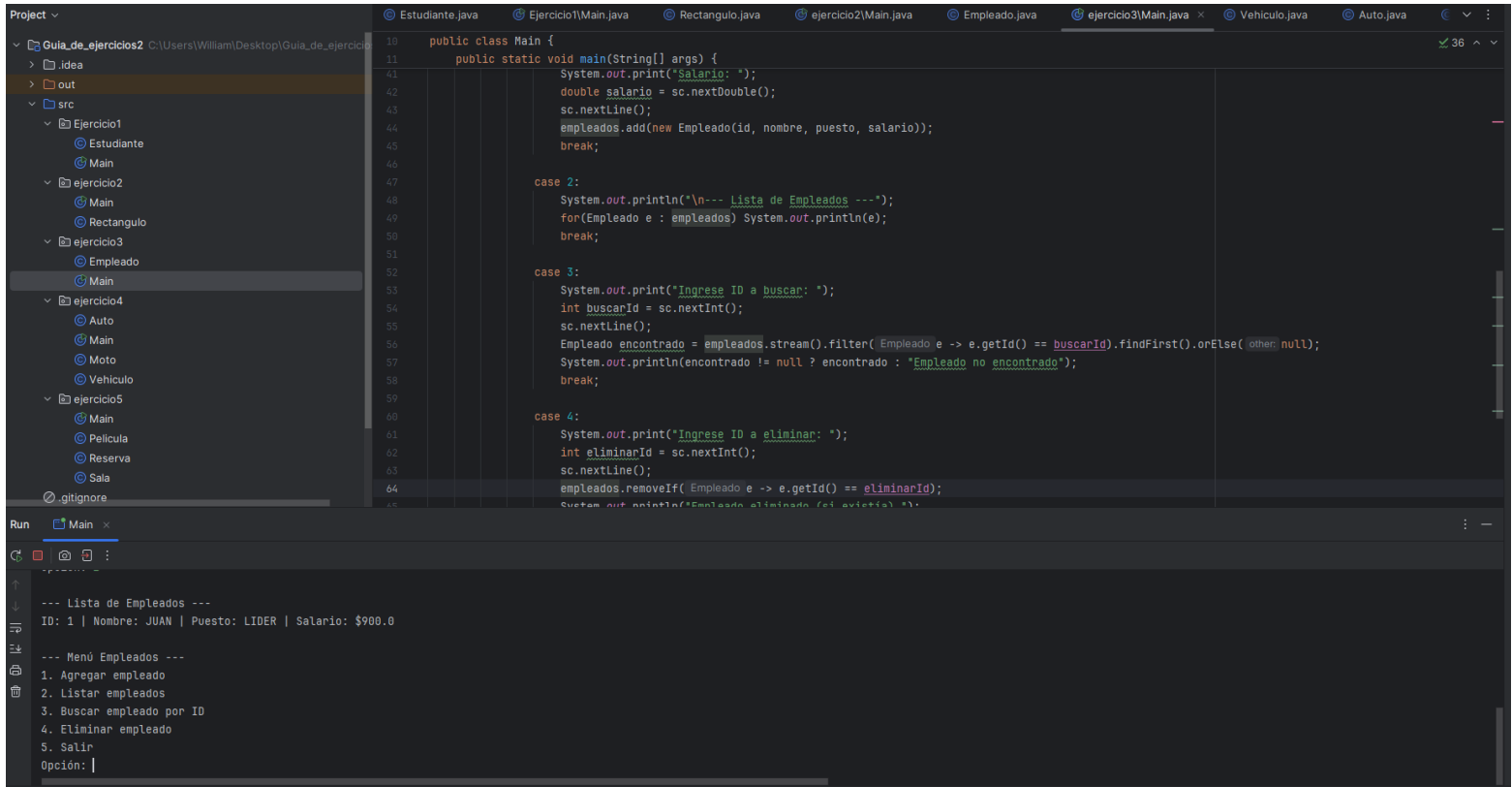
The 'Run' window shows the output of the program:

```
Base: 4
Altura: 5

Cuadrado:
Lado: 7

Resultados:
Rectángulo - Área: 20.0, Perímetro: 18.0
Cuadrado - Área: 49.0, Perímetro: 28.0
```

EJERCICIO 3



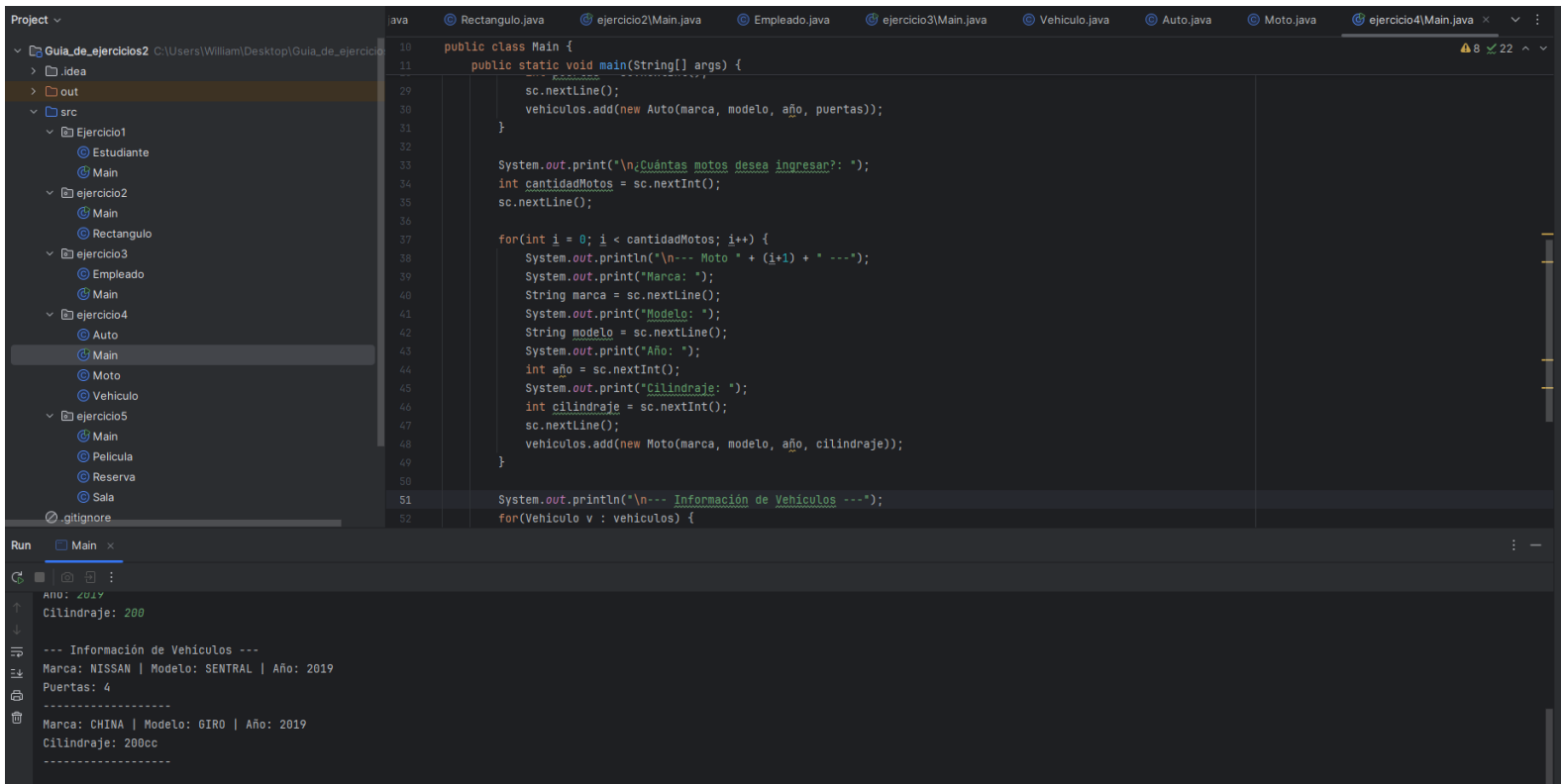
```
10 public class Main {
11     public static void main(String[] args) {
12         System.out.print("Salario: ");
13         double salario = sc.nextDouble();
14         sc.nextLine();
15         empleados.add(new Empleado(id, nombre, puesto, salario));
16         break;
17
18         case 2:
19             System.out.println("\n--- Lista de Empleados ---");
20             for(Empleado e : empleados) System.out.println(e);
21             break;
22
23         case 3:
24             System.out.print("Ingrese ID a buscar: ");
25             int buscarId = sc.nextInt();
26             sc.nextLine();
27             Empleado encontrado = empleados.stream().filter(Empleado e -> e.getId() == buscarId).findFirst().orElse(null);
28             System.out.println(encontrado != null ? encontrado : "Empleado no encontrado");
29             break;
30
31         case 4:
32             System.out.print("Ingrese ID a eliminar: ");
33             int eliminarId = sc.nextInt();
34             sc.nextLine();
35             empleados.removeIf(Empleado e -> e.getId() == eliminarId);
36             System.out.println("Empleado eliminado (si existia)");
37     }
38 }
```

Run Main

--- Lista de Empleados ---
ID: 1 | Nombre: JUAN | Puesto: LIDER | Salario: \$900.0

--- Menú Empleados ---
1. Agregar empleado
2. Listar empleados
3. Buscar empleado por ID
4. Eliminar empleado
5. Salir
Opción: |

EJERCICIO 4



```
10 public class Main {
11     public static void main(String[] args) {
12         System.out.print("Año: ");
13         int año = sc.nextInt();
14         sc.nextLine();
15         vehiculos.add(new Auto(marca, modelo, año, puertas));
16     }
17
18     System.out.print("\nCuántas motos desea ingresar?: ");
19     int cantidadMotos = sc.nextInt();
20     sc.nextLine();
21
22     for(int i = 0; i < cantidadMotos; i++) {
23         System.out.println("\n--- Moto " + (i+1) + " ---");
24         System.out.print("Marca: ");
25         String marca = sc.nextLine();
26         System.out.print("Modelo: ");
27         String modelo = sc.nextLine();
28         System.out.print("Año: ");
29         int año = sc.nextInt();
30         System.out.print("Cilindraje: ");
31         int cilindraje = sc.nextInt();
32         sc.nextLine();
33         vehiculos.add(new Moto(marca, modelo, año, cilindraje));
34     }
35
36     System.out.println("\n--- Información de Vehículos ---");
37     for(Vehiculo v : vehiculos) {
38         System.out.println(v);
39     }
40 }
```

Run Main

Año: 2019
Cilindraje: 200

--- Información de Vehículos ---
Marca: NISSAN | Modelo: SENTRAL | Año: 2019
Puertas: 4

Marca: CHINA | Modelo: GIRO | Año: 2019
Cilindraje: 200cc

EJERCICIO 5

The screenshot displays the IntelliJ IDEA IDE with a project named 'ejercicio5'. The left sidebar shows the project structure with folders for 'Ejercicio1' through 'Ejercicio5'. The 'Main' class in 'Ejercicio5' is selected. The main editor shows the code for 'Main.java', which is a Java class with a 'main' method. The code implements a menu system for a cinema system, allowing users to add movies, add rooms, make reservations, cancel reservations, show reservations, and exit. The code uses a switch statement to handle the menu options. The 'Run' button is visible at the bottom left. The output window at the bottom shows the execution of the program, displaying the menu and the user's selection of option 5, which leads to the 'Reservas' (Reservations) screen. The output shows the details of a reservation: 'Cliente: WILLIAM | Película: 01 - DEMON (ADULTO, 120 min) | Sala: 3 | Asientos: 4'.

```
10 public class Main {
11     public static void main(String[] args) {
12         // ...
49         sc.nextLine();
50         salas.add(new Sala(num, cap));
51         break;
52
53         case 3:
54             System.out.print("Nombre del cliente: ");
55             String cliente = sc.nextLine();
56
57             System.out.print("Código de película: ");
58             String codP = sc.nextLine();
59             Pelicula peli = peliculas.stream().filter( Pelicula p -> p.getCodigo().equals(codP)).findFirst().orElse( other: null);
60             if(peli == null) {
61                 System.out.println("Película no encontrada.");
62                 break;
63             }
64
65             System.out.print("Número de sala: ");
66             int numSala = sc.nextInt();
67             sc.nextLine();
68             Sala sala = salas.stream().filter( Sala s -> s.getNumero() == numSala).findFirst().orElse( other: null);
69             if(sala == null) {
70                 System.out.println("Sala no encontrada.");
71                 break;
72             }
73     }
74 }
```

Run Main x

C:\Users\William\.jdk\openjdk-23.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.3.3\lib\idea_rt.jar=55331:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.3.3\bin" -Dfile.encoding=UTF-8

--- Menú Sistema de Cine ---

1. Agregar película
2. Agregar sala
3. Realizar reserva
4. Cancelar reserva
5. Mostrar reservas
6. Salir

Opción: 5

--- Reservas ---

Cliente: WILLIAM | Película: 01 - DEMON (ADULTO, 120 min) | Sala: 3 | Asientos: 4

conclusión

Al finalizar este conjunto de ejercicios, queda en evidencia cómo la **programación orientada a objetos** no se limita a un conjunto de reglas, sino que constituye una forma estructurada y eficiente de resolver problemas. Cada práctica permitió dar un paso más en la comprensión de conceptos clave como **el encapsulamiento, la herencia, el uso de colecciones o la construcción de sistemas más integrados**.

Más allá del aspecto técnico, lo más valioso fue aprender a pensar en términos de objetos y responsabilidades, diseñando clases que reflejan entidades del mundo real y que interactúan entre sí de manera coherente. Esto refuerza la importancia de un buen diseño inicial, ya que un programa bien estructurado no solo funciona, sino que también es fácil de mantener y ampliar.

Finalmente, el desarrollo del **mini sistema de reservas de cine** sirvió como una demostración clara de que, combinando los distintos elementos aprendidos, es posible construir aplicaciones completas y funcionales. En conjunto, esta experiencia marca una base sólida para enfrentar retos más complejos en el futuro, aplicando la lógica, el orden y la creatividad que la programación orientada a objetos exige.